

Scratch 与 EV3 融合控制系统

可行性研究报告

——基于开源架构的实时数据采集与教育应用分析

2025 年 12 月

目录

右键目录，选择“更新域”刷新页码

- 1. 引言1
- 2. 系统架构分析2
 - 2.1 Scratch 3.0 开源架构 3
 - 2.2 Scratch 扩展系统 4
 - 2.3 EV3 开源固件生态 5
- 3. 通信协议分析6
- 4. 传感器系统与实时数据采集7
- 5. 融合系统统一架构设计8
- 6. 现有方案评估与功能对比9
- 7. 教育应用可行性评估10
- 8. 安全性、风险与实施路径11
- 9. 结论与展望12

摘要

本研究从系统架构、通信协议、传感器接口、数据流架构、融合设计、现有方案评估、教育应用可行性以及安全性与实施路径八个维度，对 Scratch 3.0 与 LEGO MINDSTORMS EV3 深度整合的技术可行性与教育价值进行了系统性评估。Scratch 3.0 作为全球规模最大的儿童图形化编程平台，累计注册用户超过 1.4 亿；EV3 作为 STEM 教育领域应用最广泛的机器人平台，于 2021 年 6 月正式停产，官方软件支持终止后留下巨大的教育设备利用空白。本研究提出的四层融合架构（前端交互层→虚拟机层→桥接服务层→硬件抽象层）以 TurboWarp Unsandboxed 扩展模式消除沙箱延迟，以 Python asyncio + WebSocket Server + JSON-RPC 2.0 构建 Bridge Service 替代已弃用的 Scratch Link，通过 ev3dev sysfs 接口实现对 50 余种传感器的完整驱动支持。研究进一步揭示三个跨维度战略机遇：Scratch Link 弃用与 EV3 停产的时间巧合创造了无向后兼容约束的“协议真空”创新窗口；物理计算教育领域的数据标准空白为定义“教育物理数据”新品类提供了学术发表机会；三层架构本身构成计算思维的层次化教学隐喻。项目预计开发工作量 20–31 人月，单教室部署成本\$1,480–\$5,040，建议采用“开源核心+商业服务”的双轨策略推进实施。

1. 引言

1.1 研究背景

1.1.1 Scratch 3.0 开源架构与硬件扩展的理论基础

Scratch 3.0 是全球规模最大的儿童图形化编程教育平台。截至 2024 年底，其社区累计注册用户超过 1.4 亿，年内新增用户 2,150 万，覆盖全球 200 余个国家和地区。该平台由麻省理工学院媒体实验室终身幼儿园团队开发维护，自 2019 年正式发布以来完成了从 Adobe Flash 到 HTML5 技术栈的全面迁移，采用 React 与 Redux 构建前端界面，以 Google Blockly 为基础重构积木系统，形成了高度模块化的开源架构。

Scratch 3.0 的技术架构由五个核心模块构成：scratch-gui（React 组件集合，负责编辑器界面与状态管理）、scratch-vm（Virtual Machine，虚拟机引擎，负责构造和维护抽象语法树 AST 并执行程序）、scratch-blocks（基于 Blockly 的积木渲染与交互系统）、scratch-render（WebGL 舞台渲染器）以及 scratch-audio（音频处理模块）。其中 scratch-vm 通过 blockListener 监听 scratch-blocks 工作区的事件来构建和维护 AST 状态，借助 Sequencer 组件实现协作式多任务调度。ExtensionManager 组件通过 builtinExtensions 对象管理扩展注册，支持 Core、Team、Official 和 Unofficial 四种扩展类型，这为硬件扩展在理论层面提供了完整的接口支持。

1.1.2 LEGO EV3 在 STEM 教育中的广泛应用与官方停产后的社区生态

LEGO MINDSTORMS EV3（以下简称 EV3）于 2013 年正式发布，采用 Texas Instruments AM1808 ARM9 处理器（300MHz），配备 64MB DDR RAM，通过 4 个智能传感器端口和 4 个电机端口支持丰富的外设扩展。在十余年的教育实践中，EV3 积累了庞大的课程资源体系，Carnegie Mellon Robotics Academy 的教学资源被全球数千间教室采用。

2021 年 6 月 30 日，LEGO Education 正式宣布 EV3 全球停产。这一决策对教育机构产生了深远影响：原有的 EV3 Desktop 编程软件因不兼容 macOS Catalina 而被迫停用，替代方案 EV3 Classroom 虽基于 Scratch 积木语法，但缺少数据记录、文件访问和第三方传感器支持等关键功能。与官方支持终止形成鲜明对照的是，EV3 开源社区展现出强劲的自我延续能力。ev3dev 项目——基于 Debian Linux 的完整操作系统——在 GitHub 上持续活跃更新，其内核驱动模块和构建工具在 2025 年均有定期提交记录，支持超过 51,000 个 Debian 软件包，通过 sysfs 文件系统为电机、传感器等硬件提供标准化接口。Pybricks 项目基于 MicroPython 为 EV3 提供高效的运行时环境；Open Roberta Lab 由 Fraunhofer IAIS 维护，提供 NEPO 积木语言和 Python/Java/C 代码生成能力。

1.1.3 现有 Scratch Link 方案的功能局限与深度整合需求

Scratch 3.0 与 EV3 的官方集成路径依赖于 Scratch Link 和内置的 EV3 扩展（scratch3_ev3）。然而，该方案在功能覆盖度、平台兼容性和通信架构三个维

度均存在显著局限。功能覆盖度方面，官方 EV3 扩展仅包含 11 个积木块，仅支持触摸传感器、超声波传感器（距离读数）和颜色传感器（亮度模式），不支持陀螺仪传感器、红外传感器及颜色 RGB 值读取，电机也只能按秒数控制而无法实现精确角度定位。平台兼容性方面，Scratch Link 仅提供 Windows 和 macOS 版本。通信架构方面，现有方案的数据流呈单向命令式特征——Scratch 向 EV3 发送控制指令并接收有限读数，但不支持持续的传感器数据流推送。所有评估方案均缺少与课程深度整合的实时传感器数据采集功能，这一“能力真空”正是本研究所要回应的核心问题。

1.2 研究目的与意义

1.2.1 技术层面：评估开源替代方案的可行性

本研究的首要技术目标是系统评估以开源方案替代 Scratch Link 实现 Scratch 3.0 与 EV3 联控的可行性。研究围绕三个技术问题展开：其一，在 Scratch 3.0 扩展架构的约束条件下，设计自定义 EV3 扩展的技术路径及需解决的架构瓶颈；其二，基于 ev3dev 的 sysfs 接口与 Direct Command 协议，结合 WebSocket 或 MQTT 通信方案，能否构建稳定低延迟的双向数据通道；其三，EV3 硬件资源（300MHz ARM9 无浮点运算单元、64MB RAM）对传感器采样频率和通信带宽的约束，是否可通过算法优化在合理范围内得到缓解。

1.2.2 教育层面：分析融合系统的课程价值

本研究的教育目标是分析融合系统对课程开展和学习效果提升的潜在价值。Scratch 提供的高抽象层次可视化编程环境与 EV3 硬件平台所代表的低抽象层次物理计算之间存在显著认知鸿沟——学生在 Scratch 层编程时，底层传感器数据流动是“黑箱”；而直接操作 ev3dev 或 Pybricks 时又缺乏友好的编程界面。融合系统通过在两者之间建立桥梁，使学生能够在不同抽象层次间切换观察，这种“可见性”设计有望强化从具体到抽象的认知建构。

1.2.3 数据层面：探索实时传感器数据采集的架构路径

本研究的数据目标是探索实时传感器数据采集系统对接的架构路径。现有方案的核心缺陷在于缺乏数据管道能力——原版 EV3 Lab 软件虽内置强大的数据记录功能，但已被弃用且不支持现代操作系统。研究将评估多传感器并发数据流的采集频率（目标 10–50Hz）、数据序列化格式、本地缓存策略，以及学习与分析系统的对接路径。

1.3 研究方法与报告结构

1.3.1 多维度系统分析法

本研究采用多维度系统分析法，从架构分析、协议逆向和教育评估三个研究路径并行推进。架构分析路径聚焦 Scratch 3.0 开源架构和 EV3 开源固件生态的深度解析；协议逆向路径聚焦现有通信协议的逆向工程和替代方案设计；教育评估路径聚焦融合系统的教育适用性分析。本报告共九章，系统论述研究背景、系统架构、通信协议、传感器系统、融合架构设计、数据流管道、现有方案评估、教育应用分析以及安全性与实施路径。

2. 系统架构分析：Scratch 3.0 与 EV3 开源生态

Scratch 3.0 与 LEGO MINDSTORMS EV3 均为成熟的开源生态系统，二者的技术架构决定了深度整合的可行性边界。本章从源代码级别解析 Scratch 3.0 的三层架构（虚拟机、积木系统、界面层）及其扩展机制，并系统分析 EV3 开源固件生态的驱动框架、硬件接口与替代方案，为后续通信协议设计与集成方案评估奠定技术基础。

2.1 Scratch 3.0 开源架构深度解析

Scratch 3.0 采用模块化架构设计，核心功能分布于三个主要代码仓库：scratch-vm（Virtual Machine，虚拟机）、scratch-blocks（积木块系统）与 scratch-gui（图形用户界面）。2023 年起，Scratch Team 启动 mono-repo 迁移计划，将各包整合至 scratch-editor 仓库。

2.1.1 scratch-vm 虚拟机核心：AST 结构、运行时循环与扩展注册

scratch-vm 是 Scratch 3.0 的核心执行引擎，负责表示、运行和维护程序状态。其工作机制可概括为：通过 `blockListener` 监听 `scratch-blocks` 工作区发出的事件，构造并维护一棵抽象语法树（Abstract Syntax Tree, AST），再由运行时循环驱动执行。Runtime 是驱动整个执行循环的核心环境。ExtensionManager 是扩展注册与加载的枢纽组件。内置扩展通过 `builtinExtensions` 对象静态注册，例如 EV3 扩展的注册条目为 `ev3: () => require('../extensions/scratch3_ev3')`。对于第三方平台（如 TurboWarp），还支持通过 `vm.extensionManager.loadExtensionURL()` 在运行时动态加载扩展。

2.1.2 scratch-blocks 积木系统：基于 Blockly 的块定义与 VM 松耦合设计

scratch-blocks 构建于 Google 的 Blockly 库之上。scratch-blocks 与 Blockly 的关键区别在于不使用代码生成器（code generators）。Blockly 可将块翻译为 JavaScript、Python 等多种目标语言，而 scratch-blocks 完全依赖 Scratch VM 来解释执行块。这种设计使 scratch-blocks 与 scratch-vm 之间形成松耦合关系——一块系统仅负责编辑界面的呈现与交互，不介入执行语义。

2.1.3 scratch-gui 界面层：React 组件架构、Redux 状态管理与扩展加载流程

scratch-gui 是一组 React 组件，构成创建和运行 Scratch 3.0 项目的完整界面。项目采用 Redux 进行全局状态管理。扩展的 UI 层定义在 `src/lib/libraries/extensions/index.jsx` 中，每个扩展条目包含名称、图标、描述、硬件需求等属性。

2.2 Scratch 扩展系统规格

2.2.1 四种扩展类型对比

特	Core	Team	Official	Unofficial
---	------	------	----------	------------

性 维 度				
开 发 主 体	Scratch Team	Scratch Team	第三方开 发, Scratch Team 审 核	第三方独 立开发
沙 箱 隔 离	否 (直 接运 行)	否 (直接 运行)	是 (Worker 隔离)	是 (Worker 隔离)
性 能 / 延 迟	最优 (无延 迟)	最优	存在 Worker 通信开销	同 Official
典 型 实 例	Pen (画 笔)	micro:bit、 Video Sensing	EV3、 WeDo 2.0、 BOOST	社区自定 义扩展

表 1: Scratch 3.0 四种扩展类型对比

上表揭示了 Scratch 扩展系统的分层安全模型。Core 和 Team 扩展享有最高特权——它们在主线程直接运行，无沙箱隔离，可自由访问 VM 内部 API。Official 扩展虽然可能由第三方开发，但经 Scratch Team 审核后在沙箱中运行。Unofficial 扩展仅能在 TurboWarp 等第三方平台上运行。

2.2.2 沙箱机制分析与性能影响

Official 和 Unofficial 扩展在沙箱 (sandbox) 环境中运行，其核心隔离机制基于 Web Worker 边界。沙箱中的扩展接收到的 runtime 参数是一个异步代理

对象，所有 VM API 调用通过 `postMessage` 跨越 Worker 边界序列化传递。然而，沙箱隔离带来了显著的性能开销——一个空的 COMMAND 块在沙箱环境中执行 10 次循环需要超过 1 秒，而在非沙箱环境中同一操作可瞬时完成。

2.2.3 Unsandboxed Extension 模式的技术突破

TurboWarp 等第三方平台引入了 Unsandboxed Extension 模式，从根本上消除了沙箱的性能瓶颈。与沙箱扩展在隔离 Worker 中运行不同，Unsandboxed 扩展作为普通 script 标签在主窗口中加载，可直接访问 Scratch 内部 API，块执行是瞬时且无延迟的。这一模式同时获得了无延迟执行和完整 VM 访问的双重优势，使其成为硬件控制类扩展的理想选择。

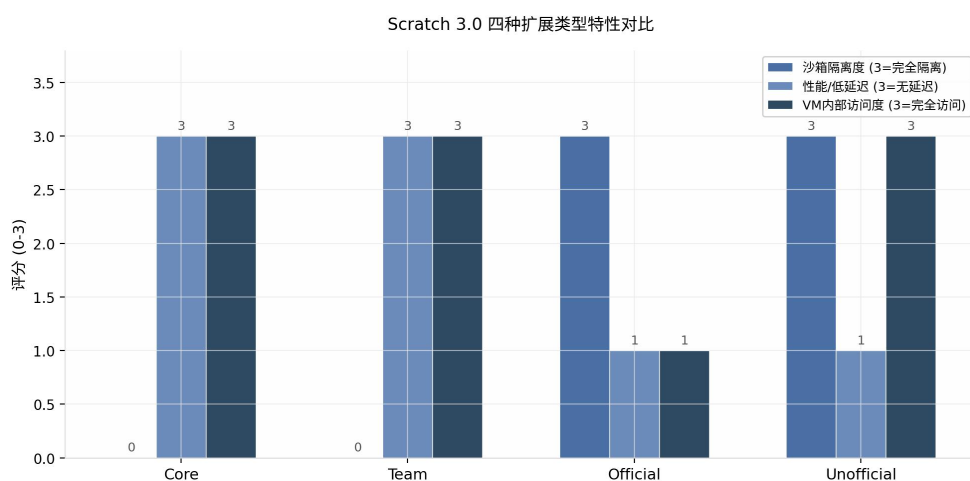


图 1: Scratch 3.0 四种扩展类型特性对比

2.3 EV3 开源固件生态

2.3.1 ev3dev 架构

ev3dev 是一个基于 Debian Linux 的完整操作系统，专为 LEGO MINDSTORMS EV3 及兼容平台设计。它通过 microSD 卡实现双启动——插入卡即运行 ev3dev，移除即恢复官方固件。ev3dev 的核心是 `lego-linux-drivers` 内核模块集合，它基于 LEGO 官方固件 LMS2012 的源代码进行 Linux 适配。电机设备映射到 `/sys/class/tacho-motor/motor<N>/`，传感器映射到 `/sys/class/lego-sensor/sensor<N>/`。这种“一切皆文件”的设计使任何能够读写文件的编程语言均可控制 EV3 硬件。

2.3.2 电机控制子系统

命令	功能描述	关键参数
run-forever	以设定速度持续运行直至收到新命令	speed_sp
run-to-abs-pos	运行至编码器绝对位置	position_sp
run-to-rel-pos	从当前位置运行相对位移	position_sp
run-timed	按指定持续时间运行	time_sp
run-direct	直接以占空比运行，可实时调整	duty_cycle_sp
stop	使用当前 stop_action 停止电机	stop_action
reset	重置所有参数至默认值并停止电机	无

表 2: ev3dev tachomotor 类支持的七种命令类型

tacho-motor 类内置两组独立的 PID 控制器，分别用于速度调节和位置保持，两组 PID 参数均可通过 sysfs 读写，使用户能够针对具体机械结构进行控制律调参。

2.3.3 传感器接口框架

lego-sensor 类是 ev3dev 中所有兼容传感器的统一接口。传感器的核心操作是模式（mode）切换——不同模式对应不同的数据输出类型和测量单位。EV3 的“智能”传感器端口支持自动识别（Auto-ID）功能。当传感器插入时，EV3 主机与传感器通过 UART 协议完成握手，初始通信以 2400 baud 进行，传感器发送自我介绍信息，确认后切换至目标波特率。

2.3.4 替代方案对比：ev3dev vs LeJOS vs Pybricks

评估维度	ev3dev	LeJOS	Pybricks
基础运行时	Debian Linux	Oracle Java SE Embedded JVM	MicroPython
内存占用	依语言而异 (Python: ~7.5MB)	~20 MB	~1.5 MB
硬件控制接口	sysfs 文件系统	lejos.hardware API	pybricks.ev3devices API

口			
电			
机			
控	内核级双	JavaMotorRegulator	PID, 参数可调整
制	PID	PID	
精			
度			
社			
区	高 (持续	中 (维护减缓)	高 (LEGO 官方支
活	维护)		持)
跃			
度			
适	高级用	Java 教育、FLL 竞	初学者、内存敏感
用	户、多语	赛	应用
场	言需求		
景			

表 3: ev3dev、LeJOS 与 Pybricks 综合评估

上述对比揭示了三个方案各自的最优适用域。ev3dev 以最大灵活性见长；Pybricks 以约 1.5MB 的运行时内存占用成为三者中资源效率最高的方案；LeJOS 因 JVM 内存开销过大，在当前 64MB RAM 限制下不推荐使用。

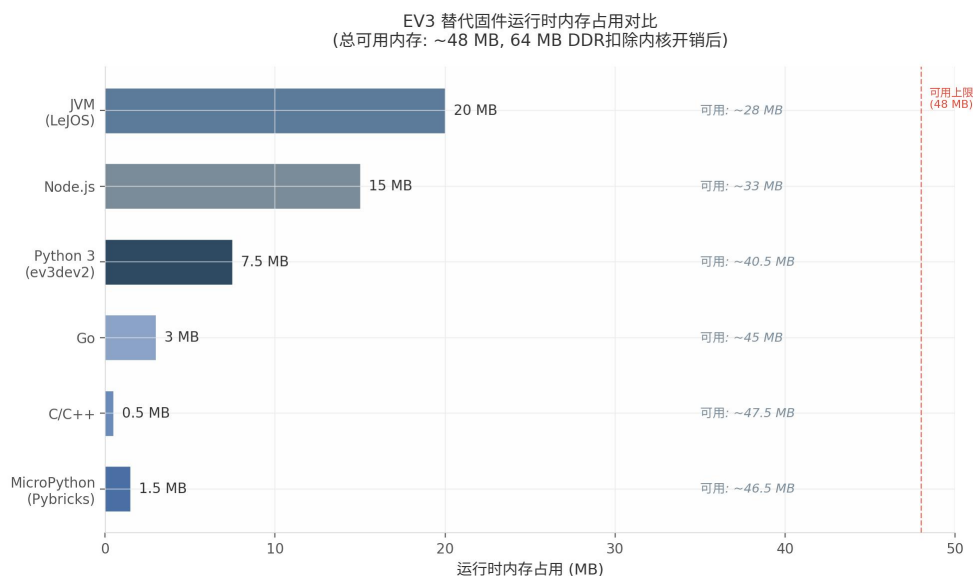


图 2: EV3 替代固件运行时内存占用与可用空间对比

3. 通信协议分析：从 Scratch Link 到统一桥接

通信协议层是实现 Scratch 与 EV3 深度融合的技术基石。Scratch Link 作为 MIT 官方开发的硬件互操作层，定义了浏览器扩展与本地蓝牙设备之间的标准通信规范；而 EV3 固件中的统一协议层则规定了主机与智能砖之间消息交换的底层格式。本章通过逆向分析 Scratch Link 的协议栈结构、系统解析 EV3 通信协议规格，并在此基础上提出统一桥接层（Bridge Service）的设计方案。

3.1 Scratch Link 协议逆向分析

3.1.1 协议栈结构

Scratch Link 采用分层协议架构实现浏览器 JavaScript 环境与本地硬件设备之间的通信桥接。其核心协议栈自顶向下包含四个层级：WebSocket 传输层、JSON-RPC 2.0 消息层、TLS/明文安全层以及蓝牙 RFCOMM 传输层。在传输层，Scratch Link 在本地启动 WebSocket 服务器，监听路径/scratch/ble（BLE 设备）和/scratch/bt（经典蓝牙设备）。浏览器中的 Scratch 3.0 扩展通过 device-manager.scratch.mit.edu 域名连接到该服务器。JSON-RPC 2.0 消息层定义了扩展

与 Scratch Link 之间的所有交互语义，支持的方法包括 discover（设备发现）、connect（设备连接）、send（数据发送）等。

3.1.2 会话状态机

Scratch Link 协议为每个 WebSocket 连接定义了严格的三阶段状态机：Initial 状态为会话起始点，仅接受 discover 请求；Discovery 状态下扫描周边设备；Connected 状态支持双向数据收发。bluepy-scratch-link 开源项目（约 556 行 Python 代码）完整实现了上述状态机。

3.1.3 安全机制演进

Scratch Link 的安全机制演进反映了教育软件在安全性与易用性之间的典型权衡。早期版本基于 TLS 1.2/1.3 协议，使用自签名证书保护 wss://device-manager.scratch.mit.edu:20110 端点。Scratch Link 2.0 将协议从 WSS 降级为 WS，端口同步从 20110 变更为 20111，从而消除了证书管理的复杂性。在本地回环通信场景下，这一降级是合理的工程权衡，显著降低了部署门槛。

3.2 EV3 通信协议规格

3.2.1 统一协议层

LEGO MINDSTORMS EV3 的通信协议设计采用了传输无关（transport-agnostic）的架构理念：无论底层使用蓝牙 SPP、USB HID 还是 WiFi TCP/IP，应用层消息格式完全统一。EV3 消息的最小头部为 7 字节，具体布局包括：Command Size（2 bytes）、Message Counter（2 bytes）、Command Type（1 byte）、Variable Header（2 bytes）和 Payload。

3.2.2 Direct Command 执行模型

Direct Command 是 EV3 协议中最核心的交互机制。从本质上看，Direct Command 是由常规字节码组成的小型程序片段，在 EV3 虚拟机的 VMTHREAD 中与正在运行的用户程序并行执行。根据 EV3 Firmware Developer Kit 的指令集规范，EV3 虚拟机共定义 185 条字节码指令，分为 14

个功能类别，涵盖传感器读取、电机控制、文件 I/O、用户界面和定时器等全部 EV3 功能。

3.2.3 System Command 功能

System Command 层提供了 15 种系统级操作，涵盖文件传输、目录管理和设备间通信等功能。WRITEMAILBOX (0x9E) 是 Scratch Link 与 EV3 通信中实际使用的主要 System Command，实现了基于命名邮箱的消息传递机制。然而，该机制虽然实现了基本的命令传递，但 JSON 序列化/反序列化在 EV3 端引入了额外的处理开销，且 Mailbox 通信为单向，EV3 无法主动推送传感器数据，这是 Scratch Link 协议无法满足实时数据采集需求的关键技术瓶颈。

3.3 协议桥接设计

3.3.1 现有开源替代方案评估

bluepy-scratch-link 由 Chris Glencross 于 2020 年发布，是一个仅 556 行的单文件 Python 脚本，完整实现了 BLESession 和 BTSession 双模式。pyscrlink 作为继任者，在 v0.2.6 版本中正式移除了蓝牙经典协议支持，移除决策的直接原因是 pybluez 项目的维护不善。针对 pybluez 的替代方案，研究表明从 Python 3.3 开始，标准库 socket 模块原生支持蓝牙 RFCOMM 连接，无需任何第三方依赖，建议作为统一桥接层的蓝牙传输基础。

3.3.2 统一桥接层 (Bridge Service) 设计

基于前述分析，本报告提出统一桥接层 (Bridge Service) 的设计方案。Bridge Service 采用 WebSocket + JSON-RPC 2.0 作为基础通信层，保持与 Scratch 扩展接口的兼容性。与 Scratch Link 不同，Bridge Service 不局限于蓝牙单一传输通道，而是在传输抽象层统一支持蓝牙经典、USB 和 WiFi 三种通信方式。协议转换映射是 Bridge Service 的核心功能：将 Scratch 积木操作直接转换为 EV3 Direct Command 字节序列，消除中间 JSON 序列化步骤；支持批量命令打包；引入传感器数据推送通道。

3.3.3 双模通信策略

评估维度	蓝牙经典 (BR/EDR SPP)	WiFi (802.11 + TCP/IP)
硬件需求	EV3 内置 蓝牙 2.1, 无需额外 硬件	需 USB WiFi dongle
典型 RTT	10–100ms (正常)	5–20ms (局 域网内)
最大并发设备数	1 主 7 从 (piconet 拓扑限制)	取决于 AP 容量, 通常 30+
教室干扰概率	38% (20– 30 台设备 密集部署)	低——WiFi 信道规划成熟
安全机制	PIN 码配 对, 无加 密	WPA2/WPA3 加密, 支持 网络隔离
适用	入门课	多设备并

用	程、单/双	发、实时数
场	设备演示	据采集
景		

表 4：蓝牙与 WiFi 双模通信策略对比

综合技术特性、教育场景需求和安全考量，本报告推荐采用蓝牙与 WiFi 的双模通信策略。入门阶段使用蓝牙模式降低技术门槛；进阶阶段切换至 WiFi 模式支持多设备并发和实时数据流。

4. 传感器系统与实时数据采集方案

EV3 平台的传感器生态系统构成了融合系统感知层的物理基础。本章从传感器技术规格、实时数据采集架构、数据存储与可视化三个维度，系统分析将 EV3 传感器数据接入数据采集系统的完整技术路径。研究表明，EV3 通过 ev3dev Linux 发行版可支持超过 50 种传感器，涵盖 5 种核心数字传感器、5 种 NXT 兼容传感器以及 20 余种第三方扩展设备，其统一的 sysfs 接口为上层应用提供了标准化的数据访问抽象。

4.1 EV3 传感器完整规格

4.1.1 核心传感器技术参数

传 感 器	工作模式	数值范 围	硬 件 采 样 率	精 度
触 摸 传 感 器	TOUCH	0-1	事 件 驱 动	二 进 制

颜色传感器	COL-REFLECT/COLOR/RGB-RAW	0-100% / 0-7 / 0-1023	1 kHz	±1% / 10-bit
超声波传感器	US-DIST-CM	0-2550 cm	~66 Hz	1 cm
陀螺仪传感器	GYRO-ANG/GYRO-RATE	±32767° / ±440°/s	1 kHz	±3°
红外传感器	IR-PROX	0-100	20 Hz	~50 ms/ 次

表 5: EV3 核心传感器技术规格对比

从表 5 的数据可以识别出三种不同的性能梯队。第一梯队为颜色传感器和陀螺仪传感器，二者均实现了 1 kHz 的硬件采样率。第二梯队为超声波传感器，其采样率受声速物理约束——在最大测量距离 250 cm 时，声波往返时间约 15 ms，对应理论最大采样率约 66 Hz。第三梯队为红外传感器，Proximity 模式约每 50 ms 进行一次新测量（20 Hz）。

4.1.2 第三方传感器生态

除 LEGO 官方传感器外，ev3dev 社区驱动支持 HiTechnic、mindsensors、Dexter Industries 等厂商的 20 余种扩展传感器。传感器端口扩展是第三方生态的关键能力——mindsensors 出品的 EV3 Sensor Multiplexer 允许将 3 个 EV3 传感器连接至单一输入端口，从而将系统支持的传感器总数从 4 个扩展到 6 个。

4.2 实时数据采集架构

4.2.1 采集层设计

传感器数据采集的采集层负责从 EV3 硬件读取原始传感器值并执行边缘预处理。ev3dev 通过 sysfs 文件系统暴露传感器接口，核心属性包括：mode（读写当前模式）、value0–value7（传感器读数）、decimals（当前模式小数位数）、poll_ms（轮询周期）等。sysfs 读取性能是采集层设计的关键约束——单次 seek+read+write 三个系统调用的总耗时约 0.885 ms，意味着理论最大读取速率约 1000 Hz。针对不同传感器物理特性，推荐采用多速率轮询策略（Multi-Rate Polling Strategy）。

4.2.2 传输层设计

传输层负责将采集的传感器数据从 EV3 推送至主机。WebSocket (RFC 6455) 被选定为首选传输协议，其全双工通信能力允许在单一 TCP 连接上实现双向异步消息传输，典型端到端延迟低于 10 ms。MQTT 作为备选方案，适用于需要发布-订阅模式和解耦架构的场景，推荐传感器遥测数据使用 QoS 0，控制命令使用 QoS 1。

4.2.3 缓冲与队列管理

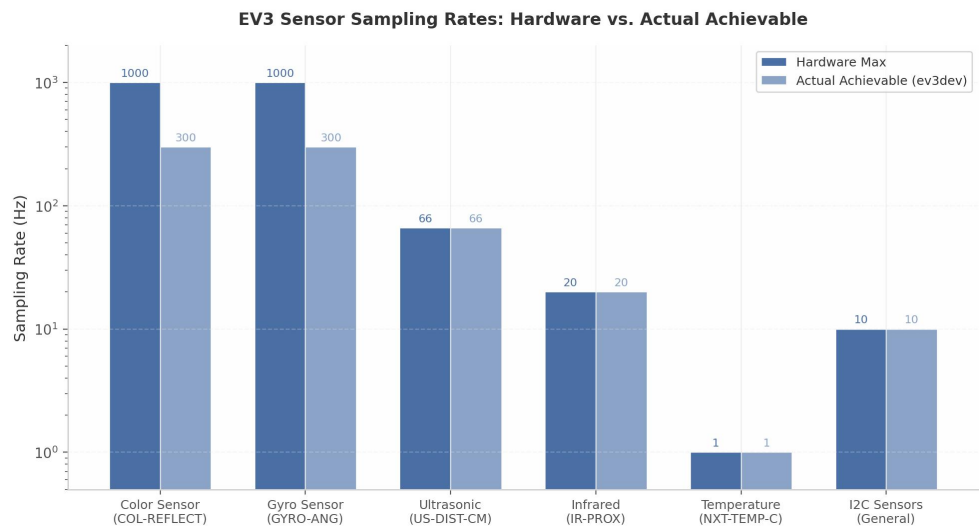


图 3：EV3 传感器硬件采样率与实际可达采样率对比

为应对网络波动和传输层瞬时拥塞，采集层与传输层之间需引入缓冲机制。环形缓冲区（Ring Buffer）是嵌入式系统中的标准选择，推荐配置为 1024 样本容量，以 100 Hz 采样率计算可缓冲约 10 秒数据。背压处理（Backpressure Handling）是流系统中不可避免的挑战，需实施自适应降级机制：正常模式下全速采样；队列填充率超过 70%时启动自适应降速；填充率超过 95%时进入丢帧模式。

4.3 数据存储与可视化

4.3.1 时序数据库选型对比

参数	InfluxDB 2.x	TimescaleDB	SQLite (+时间索引)
写入吞吐 量	280K 点/秒	200K 点/秒	~5K 点/秒
查询	~50 ms	~15 ms	~1 ms

延迟			
存储			
压缩比	10–20×	10–15×	N/A
RAM 占用			
	~150 MB	~200 MB	<1 MB
部署复杂度			
	Docker 一键部署	需 PostgreSQL 基础	零依赖 嵌入
适用场景			
	高频采集、大规模	SQL 查询、关系关联	嵌入式、轻量级

表 6: 时序数据库选型对比

综合评估表明，教育场景下推荐采用“InfluxDB + SQLite”双后端架构：InfluxDB 负责高频写入和长期存储，SQLite 承担快速查询和本地备份职责。

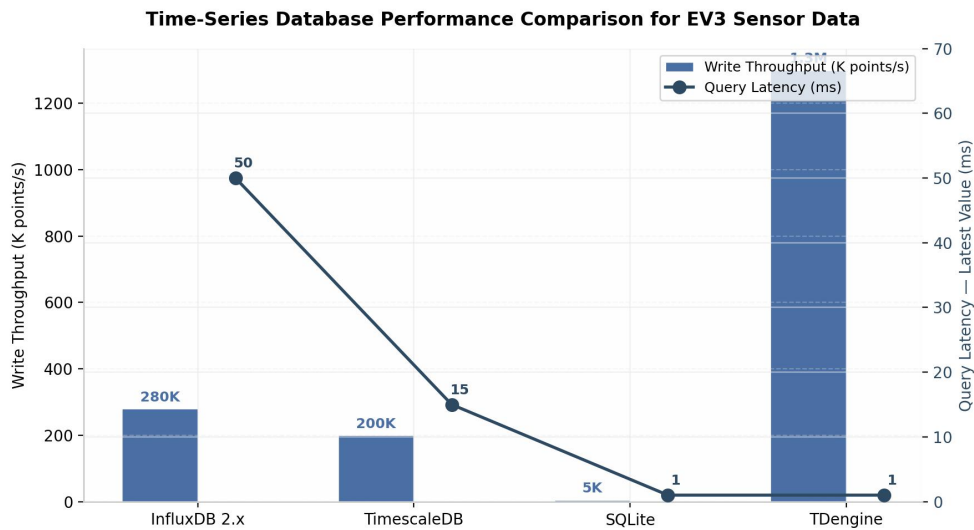


图 4: 时序数据库写入吞吐量与查询延迟对比

4.3.2 实时可视化方案

实时可视化是传感器数据价值的最终呈现层。Grafana 是 IoT 监控领域的事实标准，提供专业的实时仪表盘、阈值告警和数据探索功能。Plotly Dash 在教育场景适配性方面表现突出，作为 Python 原生框架，允许学生直接使用熟悉的 Python 语法创建交互式实时图表。对于 Scratch 内嵌可视化，独立浏览器窗口方案最为灵活，可利用完整可视化功能而不受 Scratch 安全沙箱限制。

4.3.3 采样频率优化

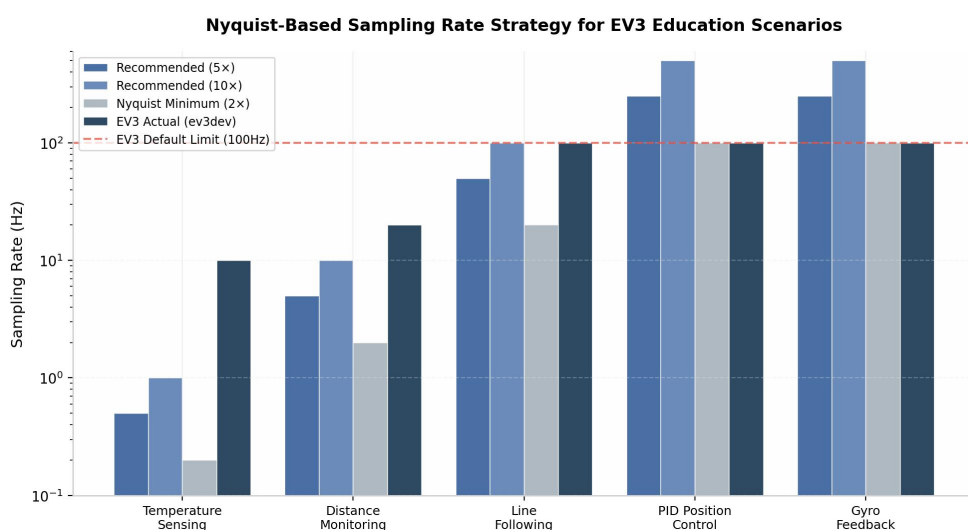


图 5：基于奈奎斯特定理的 EV3 教育场景采样率分级策略

采样频率的确定需遵循奈奎斯特-香农采样定理：为避免信号混叠，采样率必须不低于信号最高频率分量的两倍。基于该定理，EV3 教育场景的采样频率可划分为三个层级：低频观测（1–10 Hz）适用于温度监测等缓慢变化信号；中频采集（10–50 Hz）适用于距离监测、环境光测量等场景；高频反馈（100 Hz）适用于线跟踪循迹、PID 位置控制等闭环控制场景。在网络带宽层面，经 MessagePack 二进制序列化和 WebSocket 压缩双重优化后，实际带宽需求可降至约 77 Kbps，远低于各种传输方式的可用带宽。

5. 融合系统统一架构设计

前述章节分别从 Scratch 3.0 系统架构、通信协议分析和传感器与数据采集方案三个维度展开了独立的技术论证。本章将这三条分析线索汇拢，提出一套完整的 Scratch 与 EV3 融合统一架构方案。该架构的核心设计目标是：替代已弃用的 Scratch Link 中间件，实现 Scratch 前端与 EV3 硬件的深度整合，支持双向实时数据流——既保留 Scratch Blocks 可视化编程的教育亲和力，又满足联控与实时数据采集的技术需求。

5.1 总体架构设计

5.1.1 四层架构模型

融合系统采用严格分层的四级架构模型，自顶向下依次为：前端交互层（Frontend Layer）、虚拟机层（VM Layer）、桥接服务层（Bridge Service Layer）与硬件抽象层（Hardware/EV3 Backend Layer）。前端交互层基于 TurboWarp 的 Unsandboxed 扩展定制提供完整硬件控制积木块；虚拟机层执行 Scratch VM 运行时与扩展管理；桥接服务层以 Python asyncio + WebSocket Server 实现协议转换与设备管理，替代 Scratch Link；EV3 后端层以 python-ev3dev2 库和 systemd 常驻服务提供传感器数据采集循环与命令执行。

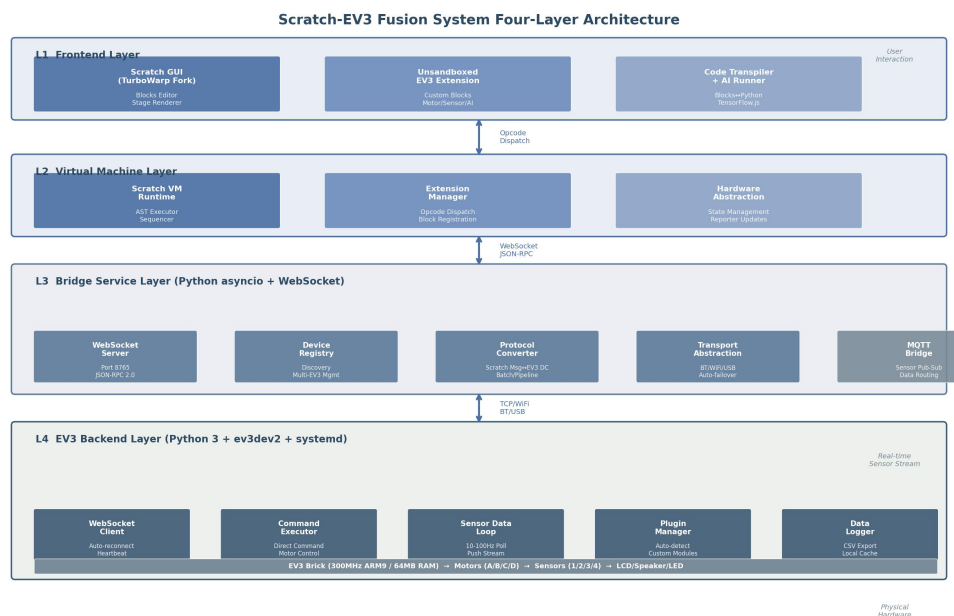


图 6: Scratch-EV3 融合系统四层架构模型

图 6 展示了四层架构的完整组件拓扑。层间通过明确定义的接口通信：

L1↔L2 通过 Opcode Dispatch, L2↔L3 通过 WebSocket JSON-RPC 2.0, L3↔L4 通过 TCP/WiFi 或蓝牙/USB。控制流（下行：Scratch→EV3）的执行路径为：用户在 Scratch GUI 中拖拽 EV3 积木块 → VM 调用扩展中对应 Opcode 处理函数 → 扩展通过 WebSocket 向 Bridge Service 发送 JSON-RPC 请求 → Bridge Service 将 Scratch 消息转换为 EV3 Direct Command 字节码 → 通过 TCP/WiFi 或蓝牙发送至 EV3 后端 → 后端调用 ev3dev2 API 或直接写入 sysfs 执行硬件操作。数据流（上行：EV3→Scratch）则沿反向路径，EV3 后端以配置频率轮询传感器，传感器数据通过 WebSocket 推送至 Bridge Service，缓存并广播给已连接的 Scratch 客户端。

5.1.2 前端层：基于 TurboWarp 的 Unsandboxed 扩展定制

前端层的设计决策基于第 2 章对 Scratch 扩展系统的深入分析。Scratch 官方 EV3 扩展采用 Official 类型，在 Worker 沙箱中运行，每次块执行至少延迟 1 帧（约 16.7ms@60fps），对于电机实时调速等高频控制场景，这一延迟是不可接受的。TurboWarp 等第三方平台引入的 Unsandboxed Extension 模式从根本

上消除了沙箱的性能瓶颈——扩展作为普通 script 标签在主窗口中加载，可直接访问 Scratch 内部 API，块执行是瞬时且无延迟的。

本方案的前端层基于 TurboWarp Fork 构建，在 scratch-vm/src/extensions/ 目录下创建自定义 EV3 扩展模块（命名为 scratch3_ev3pro）。扩展积木块的设计聚焦于弥补官方扩展的功能缺失：电机控制方面，新增精确定位电机块、相对位置移动块和坦克驱动块；传感器读取方面，新增完整传感器读取块、RGB 颜色详细读取块和陀螺仪角度读取块；数据采集方面，新增批量数据采集启动块、采集结果获取块和数据阈值事件触发块；AI 集成方面，新增 AI 模型推理块和 AI 控制电机块。

5.1.3 桥接服务层：Python asyncio + WebSocket Server + JSON-RPC 2.0

桥接服务层是四层架构的核心创新组件，其设计目标明确替代 Scratch Link 并解决其三个根本性局限。Bridge Service 采用 Python asyncio 和 websockets 库实现，利用异步 I/O 模型处理高并发连接——websockets 库在生产环境中可支持约 10,000 并发连接/核心，对于教室级部署（最多 30–40 个连接）性能冗余充足。WebSocket 服务器监听端口 8765，协议设计保持与 Scratch Link 接口风格的兼容性，同时在语义层面大幅扩展：新增 startDataStream/stopDataStream 方法启动/停止传感器数据流推送、executeDirectCommand 方法暴露底层 Direct Command 执行接口、getDeviceCapabilities 方法查询设备能力集。

5.2 EV3 后端服务设计

5.2.1 常驻服务架构

EV3 后端服务是四层架构的底层执行节点，常驻于 EV3 智能砖上。服务基于 python-ev3dev2 库构建，采用 systemd 守护进程管理，确保 EV3 开机后自动启动并在崩溃后自动恢复。传感器数据采集循环是后端服务的核心功能，基于 ev3dev 官方文档的 Daisy Chain 性能数据，单台 EV3 在 4 电机+4 传感器满配置下，所有数值约每 100ms 更新一次（即 10Hz）。本方案的数据循环采用

多速率轮询策略：颜色传感器和陀螺仪传感器配置为 50–100Hz，超声波传感器配置为 10–20Hz，触摸传感器配置为 50Hz，红外传感器配置为 20Hz。

5.2.2 双模式运行

融合系统的教育价值很大程度上来源于其支持的双模式运行能力——学生在 Scratch Blocks 可视化编程（教育入门模式）与 Python 直接编程（高级应用模式）之间自由切换。模式 A 面向 8–14 岁入门学生，提供完整的 EV3 控制积木块、实时传感器数值显示和数据可视化图表。模式 B 面向 14 岁以上学生和竞赛开发者，提供完整的 ev3dev2 Python API 访问、Jupyter Notebook 交互式开发环境和 ipywidgets 实时控件。两种模式之间的切换通过代码转换器（Transpiler）实现。

5.2.3 AI 功能集成

AI 功能集成是融合系统区别于现有方案的关键差异化能力。Scratch 生态已集成多种 AI 扩展，包括 Google Teachable Machine（支持图像、声音、姿势三种分类模型）、Image Trainer、FaceMesh、HandPose 和 PoseNet，这些扩展均使用 TensorFlow.js 在浏览器内运行。本方案的设计思路是复用上述成熟的 Scratch AI 扩展生态，将其与 EV3 传感器数据流进行深度融合。AI 集成的数据流架构如下：EV3 传感器数据通过 Bridge Service 推送至 Scratch VM，VM 扩展将传感器值存入 Scratch 变量；AI 扩展读取这些变量作为模型输入，在浏览器内的 TensorFlow.js 运行时执行推理；推理结果通过 Scratch 变量传递回 EV3 扩展，后者将其转换为电机控制命令或 LED 显示指令下发至 EV3。

5.3 规模化部署架构

5.3.1 教室级设备管理

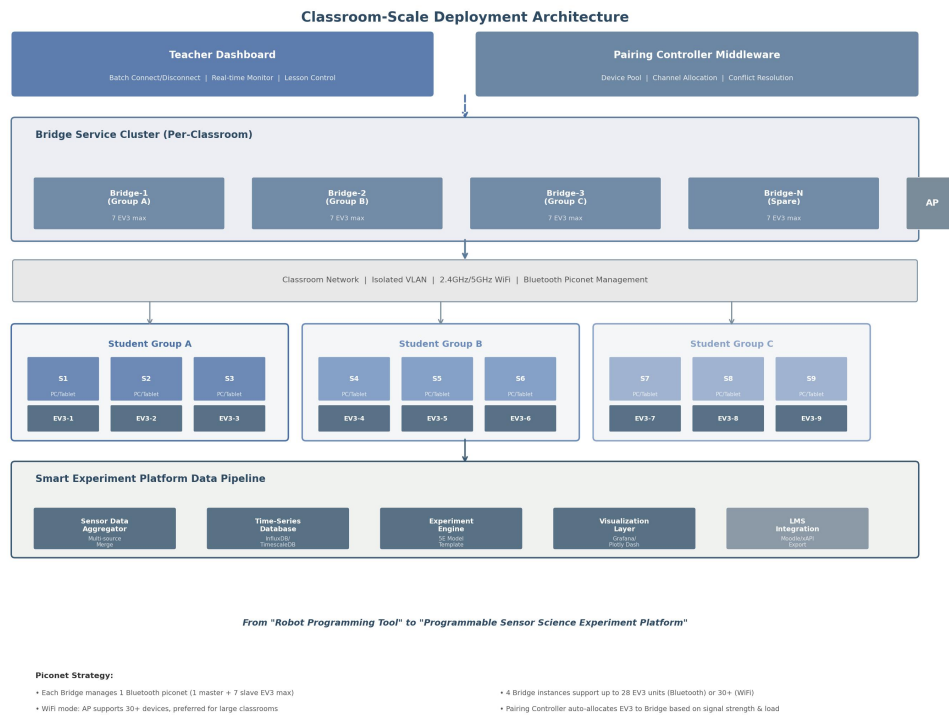


图 7：教室级规模化部署架构

图 7 展示了面向 20–30 台 EV3 同时运行的教室级部署架构。该架构的核心挑战源于蓝牙的 piconet 拓扑限制——一个蓝牙主设备最多同时连接 7 个从设备，而典型教室需要支持 20–30 组学生。本方案采用“分区+双模”策略解决规模化部署瓶颈：桥接服务集群按学生组划分为多个 Bridge 实例，每个 Bridge 实例管理一个独立的蓝牙 piconet。配对控制器中间件承担三项关键功能：设备池维护、信道分配和冲突解决。教师 Dashboard 提供教室级设备管理的可视化界面，涵盖批量连接/断开、实时监控面板、课程控制和实验数据汇总。

5.3.2 “智能实验平台”范式

本章前面所述的四层架构和规模化部署方案，其最终指向不仅是技术整合——而是面向 STEM 教育的一个新品类定义。现有 Scratch+EV3 的应用主要

是“机器人编程”（控制导向），而融合系统与数据采集系统深度整合后，EV3 从“机器人”变成了“可编程传感器平台”，Scratch 从“游戏动画工具”变成了“实验设计界面”——两者都发生了质变。传感器数据管道五层架构（采集→传输→处理→存储→可视化）与 Scratch Blocks 编程界面深度融合后，学生可以通过拖拽积木块的方式设计完整的科学实验：设定采样频率和持续时间 → 配置多传感器同步采集 → 实时观察数据曲线 → 导出数据进行统计分析 → 生成实验报告。

6. 现有方案评估与功能对比

在融合系统的架构设计之前，有必要对当前可用的 Scratch 与 EV3 整合方案进行系统性评估。截至 2025 年，市场上存在六款主要方案，涵盖从官方扩展到开源社区项目的多种技术路线。本章从传感器覆盖度、编程模式、平台兼容性、实时数据流、开源程度、教育适配性和维护状态七个维度展开量化评估。

6.1 主流方案全面评估

6.1.1 官方 Scratch EV3 扩展：功能严重受限

Scratch 3.0 内置的 LEGO MINDSTORMS EV3 扩展是目前最广为人知的整合方案，但其功能覆盖度远低于教育实践所需。根据 Scratch Wiki 官方文档，该扩展仅提供 11 个积木块，电机控制仅支持按秒数运行和功率百分比设置，不支持按精确角度定位；传感器方面仅覆盖触摸传感器、超声波传感器和颜色传感器的亮度模式，陀螺仪传感器和红外传感器完全未获支持。综合评分为 3.0/10，仅适用于最基础的入门演示场景。

6.1.2 Open Roberta Lab：教育场景下的开源最佳实践

Open Roberta Lab 由德国弗劳恩霍夫智能分析与信息系统研究所（Fraunhofer IAIS）开发，采用 NEPO 块状语言，其语法设计类似于 Scratch。该平台完整支持 EV3 全部五款核心传感器，提供 2D 虚拟仿真环境，连接方式支持 WiFi 和 USB。综合评分为 8.4/10，在现有方案中位列教育适配性首位。

6.1.3 Pybricks: 功能完整但缺乏块编程支持

Pybricks 是一款基于 MicroPython 的开源固件。在传感器支持与控制精度方面表现卓越，为 EV3 的 Color Sensor、Touch Sensor、Infrared Sensor、Gyro Sensor、Ultrasonic Sensor 以及 Large/Medium Motor 均提供了完整驱动支持。电机控制方面内置 PID 控制器。核心短板在于编程门槛：要求用户具备 MicroPython 编程能力。综合评分为 8.3/10。

6.2 功能对比矩阵

6.2.1 七维度对比分析

为系统性比较各方案的功能差异，本研究从七个维度建立了量化评估框架。图 6-1 展示了六款方案的综合评分对比。Open Roberta Lab (8.4/10) 与 Pybricks (8.3/10) 构成第一梯队，两者分别在教育适配性和技术功能完整性上占据优势；MakeCode EV3 (7.7/10) 位居第二梯队；官方 Scratch 3.0 EV3 扩展 (3.0/10) 显著落后于其他方案。

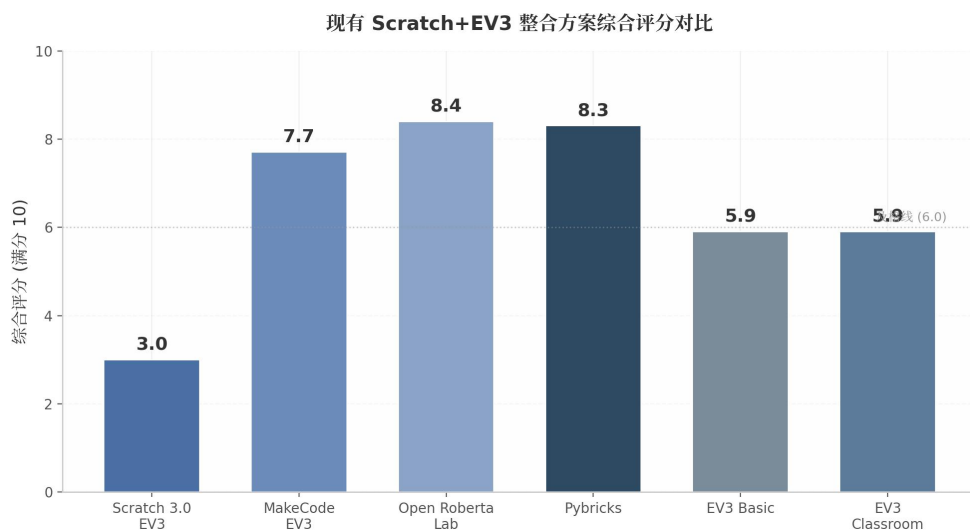


图 8: 现有 Scratch+EV3 整合方案综合评分对比

图 6-2 以热力图形式直观呈现六款方案在七个评估维度上的得分分布，深色区域代表高分优势，浅色区域标识功能短板。Pybricks 在“电机控制精度”和“开源程度”两个维度上获得满分，但在“编程模式”维度仅获 6/10。官方 Scratch 3.0 扩展在“开源程度”维度为 0/10，这是其总分显著落后的核心原因之一。

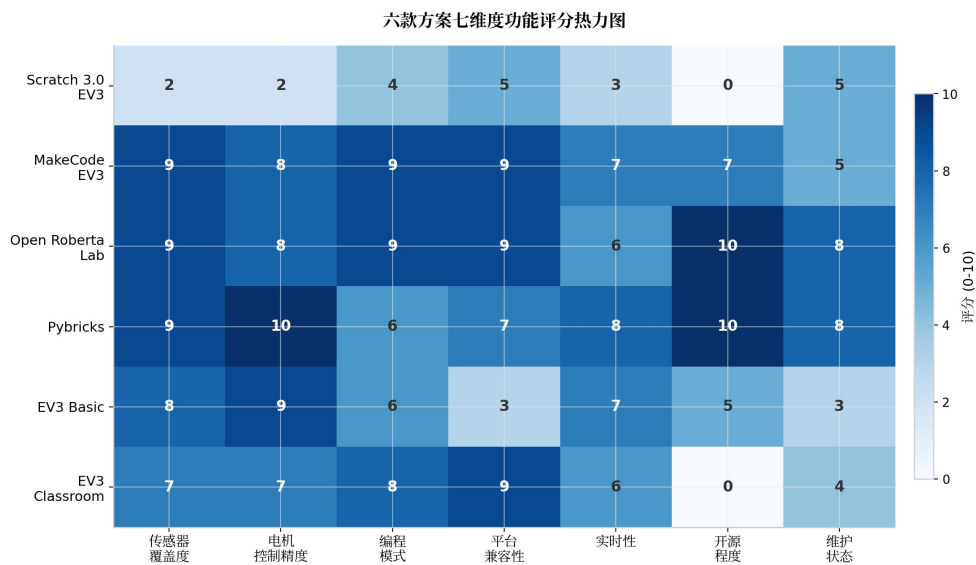


图 9：六款方案七维度功能评分热力图

6.2.2 用户需求差距分析

基于上述七维度对比，本研究进一步从用户视角分析现有方案的功能缺口。关键发现是：没有任何现有方案能够同时满足“完整传感器支持”、“可视化块编程”和“实时传感器数据采集”这三项核心需求。这一“三位一体”需求缺口正是本融合系统的核心定位空间。在实时数据采集维度，所有六款方案均存在结构性缺失；在科学实验整合维度，现有方案的设计目标均定位为“编程入门工具”而非“科学实验平台”；在多设备协作维度，现有方案均基于“一对一”的连接模型。

7. 教育应用可行性评估

教育应用可行性是评估 Scratch 与 EV3 融合系统价值的核心维度。本章从教育理论框架、课程整合设计和教学实践可行性三个层面展开论证，回答教育工作者最关注的问题：该融合系统是否有利于课程开展，能否有效促进学生计算思维（Computational Thinking, CT）和科学素养的发展。

7.1 教育理论框架

7.1.1 三大理论支撑：从建构主义到体验学习的理论链条

融合系统的教育价值植根于三大经典学习理论的交叉验证。第一条理论线索始于 Papert 的建构主义 (Constructionism) ——学习者通过创建可分享的有形物品来实现最佳知识建构。Scratch 的设计哲学直接源于这一理论，其积木式界面消除了语法复杂性，使学习者得以专注于项目的逻辑与创意设计。融合系统同时继承 Scratch 的数字建构环境与 EV3 的物理建构能力，天然契合建构主义的核心主张。

第二条线索是 Bybee 于 1997 年提出的 5E 探究式教学模型 (Engage-Explore-Explain-Elaborate-Evaluate)。研究表明，基于 5E 模型的机器人科学课程能有效促进学生对科学概念的三维学习，特别是在 NGSS 框架下。融合系统的“三明治架构”恰好映射 5E 模型的递进结构。

第三条线索是 Kolb 于 1984 年提出的体验学习循环 (Concrete Experience→Reflective Observation→Abstract Conceptualization→Active Experimentation)。Govender 和 Govender (2023) 的研究表明，将每个工作坊设计为一个 Kolb 微循环能有效促进学生编程概念的渐进式发展。融合系统通过实时传感器数据流动为学生提供“具体经验”，通过代码调试和数据分析引导“反思观察”，进而在 Scratch 抽象层完成“抽象概念化”，最终通过修改程序重新部署实现“主动实验”——完整覆盖了 Kolb 循环的四个阶段。

7.1.2 计算思维培养框架：四维 CT 培养映射

计算思维作为融合系统的核心教育目标，需要明确的培养维度定义。Brennan 和 Resnick (2012) 提出了 Scratch 教育领域最广泛使用的 CT 框架，包含计算概念、计算实践和计算视角三个维度。融合系统在计算思维四维框架——分解、模式识别、抽象和算法设计——中展现出超越单一工具的培养优势。

图7-2 融合系统计算思维四维培养框架映射

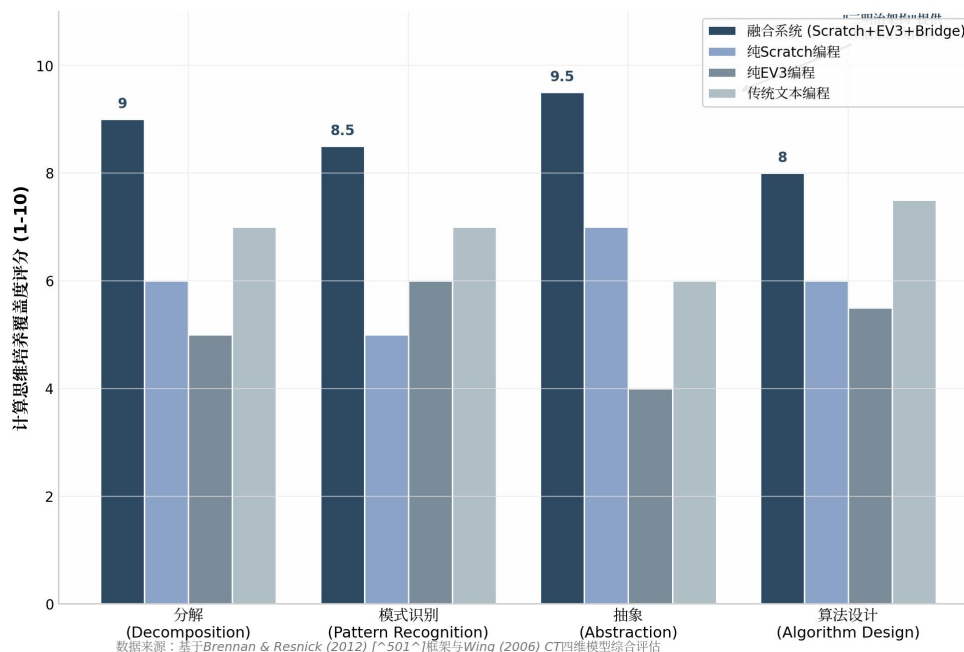


图 10: 融合系统计算思维四维培养框架映射

如图 10 所示，融合系统在“抽象”维度的培养覆盖度达到 9.5/10，显著高于纯 Scratch 编程 (7.0/10)、纯 EV3 编程 (4.0/10) 和传统文本编程

(6.0/10)。传统 Scratch 编程囿于虚拟环境，无法提供物理世界的真实数据反馈；纯 EV3 编程又过于底层，抽象层次不足；融合系统恰好填补了两者之间的认知鸿沟。

7.1.3 实证研究证据

元分析证据为融合系统的教育效果提供了强有力的定量支撑。一项涵盖 22 项元分析的伞形综述发现，STEM 教育在小学阶段的平均效应量为 $d = 0.48$

(中等效应)，初中阶段为 $d = 0.44$ ，高中阶段为 $d = 0.56$ ，大学及以上阶段为 $d = 0.75$ 。更为直接的证据来自 Arduino 与 Scratch 整合的专项元分析：该研究综合了 11 项定量研究、29 个独特效应量，发现整合干预对学生 STEM 学业成绩和 STEM 态度有整体正向效应 ($d = 0.67$, CI: 0.40-0.95)。研究者将这一差异归因于“物理计算的可触知性”——学生通过操作实体硬件获得即时、具体的反馈，从而增强了学习投入度和概念理解深度。

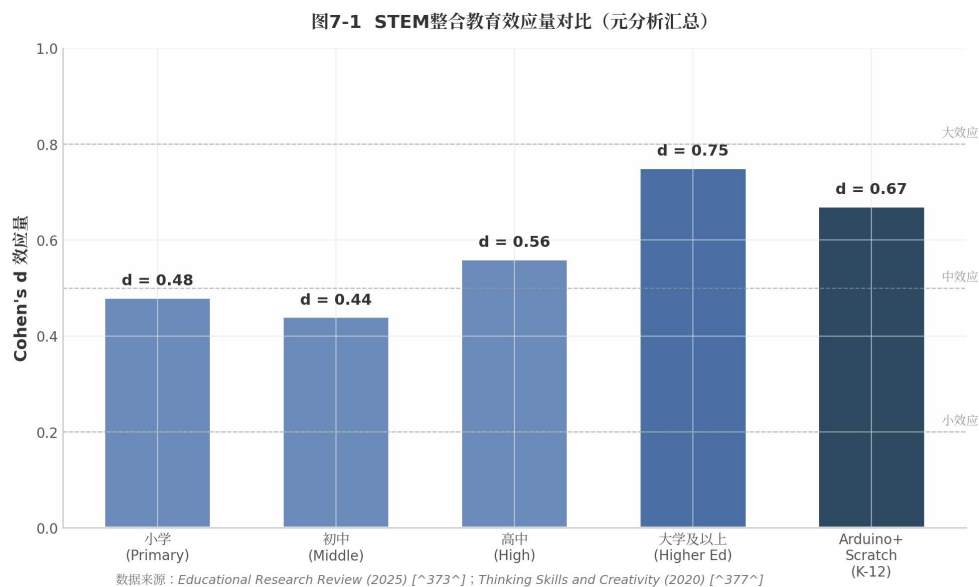


图 11：STEM 整合教育效应量对比（元分析汇总）

7.2 课程整合设计

7.2.1 三阶段课程模块

基于上述理论框架和实证证据，融合系统可设计为三阶段递进式课程模块。入门模块（6-8 课时）面向小学高年级，目标是建立 Scratch 与 EV3 的基础连接概念，核心概念覆盖序列、事件和条件判断。进阶模块（8-10 课时）面向初中阶段，重点转向传感器数据采集实验和反馈控制，核心转变是从“控制导向”到“数据导向”。高级模块（10-12 课时）面向初中高年级和高中学生，聚焦复杂系统设计和人工智能初步。三个模块均采用 4P 教学法（Projects 项目驱动、Passion 兴趣驱动、Peers 协作学习、Play 游戏化探索）和 Use-Modify-Create 学习进阶模型。

7.2.2 课程标准对接

融合系统的课程设计可直接对接中国 2022 信息科技课程标准、美国 NGSS 和 ISTE 学生标准三大权威框架。中国 2022 信息科技课程标准第四学段（7-9 年级）的“物联网实践与探索”模块与融合系统的契合度最高——该模块明确要求学生能“设计并搭建具有数据采集、实时传输和简单控制功能的简易物联网系统”，而 EV3 的传感器系统正是连接物理世界与数字世界的典型媒介。NGSS

的 ETS 标准强调三维学习，融合系统的数据采集和反馈控制能力直接服务于“科学与工程实践”维度。ISTE 学生标准 1.5 明确要求“学生使用算法思维开发和测试自动化解决方案”。

7.2.3 学习成效评估

有效的学习成效评估是课程整合成功的保障。融合系统的评估体系采用三维互补工具组合：CTt 计算思维测评（总结性能力测试）、Bebras 算法思维挑战（情境化迁移评估）和 Dr.Scratch 代码质量分析（形成性迭代工具）。三种核心工具之间存在中等强度正相关（ $r = 0.44-0.53$ ），表明它们从不同视角评估同一心理构念——计算思维。

7.3 教学实践可行性

7.3.1 课堂部署方案

课堂部署的可行性取决于设备配比、空间布局和网络基础设施三个要素。设备配比方面，研究表明 3-4 人小组是最优配置，但融合系统的“智能实验平台”定位建议采用 2-3 人/组。空间布局应划分为编程区、实验区和展示区三个功能区域。网络基础设施是规模化部署的关键考量——当一间教室 20-30 组学生同时使用蓝牙连接时，干扰概率可达 38%，因此架构设计应支持双模通信。

7.3.2 教师培训路径

教师培训是融合系统成功实施的关键因素。建议设计三阶段培训模型：第一阶段“技术操作”（8 小时）聚焦融合系统的基本操作；第二阶段“课程设计”（16 小时）深入教学理论与课程设计；第三阶段“数据素养”（8 小时）面向融合系统的核心差异化能力——传感器数据采集与分析。三个阶段总计 32 小时的培训负荷与典型的教师暑期研修项目相当，具有现实可行性。

8. 安全性、风险与实施路径

8.1 安全性分析

8.1.1 蓝牙安全风险评估

LEGO EV3 采用蓝牙 2.1+EDR（经典蓝牙，BR/EDR）进行无线通信，其安全特性与当代蓝牙低功耗（BLE）标准存在显著差距。研究表明，EV3 的配对过程默认采用 Secure Simple Pairing（SSP）中的 Just Works 模式，不提供中间人（MITM）攻击防护。CVE-2019-9506（KNOB 攻击）允许攻击者将会话加密密钥的熵值强制降至 1 字节；CVE-2020-10135（BIAS 攻击）则允许未经认证的邻近攻击者冒充先前配对过的设备。

然而，上述技术漏洞在教育教室环境中的实际可利用性受到“半可信物理空间”特性的显著制约。攻击者需同时满足三项条件：物理距离在蓝牙 10 米范围内、在授课时段内完成操作、且突破教室网络隔离。这种“教室免疫”效应使得实际攻击面大幅收窄——系统的安全威胁模型应以“防止学生误操作和设备间干扰”为核心目标。

8.1.2 网络安全对策

针对 WebSocket 通信层的安全风险，系统应实施三层防护架构。第一层为 WebSocket Origin 验证，防止跨站 WebSocket 劫持（CSWSH）攻击。第二层为专用隔离 WiFi/VLAN，K-12 学校网络应至少实施 4 个 VLAN：Staff & Admin VLAN、Student VLAN、IoT & Infrastructure VLAN、Guest VLAN。第三层为学生数据匿名化处理，传感器数据应尽可能在本地处理，避免将原始学生数据传输至云端。

8.1.3 系统稳定性设计

系统稳定性是教室环境中保障教学连续性的核心要求。蓝牙连接层的稳定性设计应包含断线重连机制。WebSocket 连接保活机制是维持通信稳定的关键——心跳间隔建议设置为 25-55 秒，检测超时阈值设定为连续 2-3 次未响应

后宣告连接死亡，重连过程应采用指数退避策略。EV3 端服务进程崩溃恢复可通过 Linux 系统内置的 systemd 或 init 脚本实现自动重启。

8.2 技术风险评估

8.2.1 关键风险矩阵

基于系统架构、通信协议、硬件特性和外部环境的多维度分析，本节识别出 10 项关键技术风险。两个“极高风险”项：蓝牙连接不稳定（pybluez 停止维护）和 EV3 硬件获取困难（LEGO 于 2021 年停产）。三个“高风险”项：多设备蓝牙干扰、学生数据泄露和网络横向移动。中等风险项以软件维护和生态变迁为主，可通过社区参与和架构抽象进行缓解。

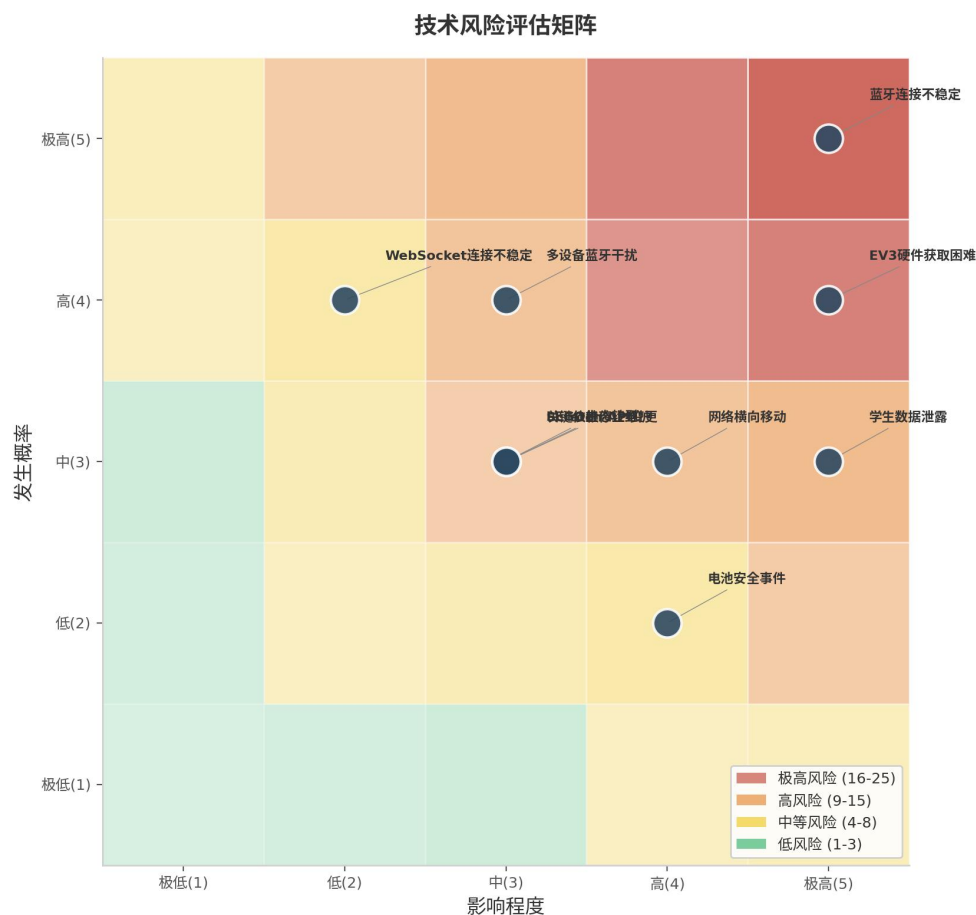


图 12: 技术风险评估矩阵

8.2.2 缓解策略

针对极高风险项，系统采用多层次的缓解策略组合。蓝牙连接不稳定的核心缓解路径是绕过 pybluez，直接使用 Python 原生 socket 模块创建 RFCOMM 连接，彻底消除对第三方维护状态的依赖。EV3 硬件获取风险的缓解策略包含三个层次：优先利用学校现有 EV3 设备（零增量成本），通过 ev3dev 开源社区延长设备使用寿命，以及评估向 SPIKE Prime 或 Raspberry Pi+LEGO 平台的迁移路径。

8.3 实施路径与成本

8.3.1 五阶段开发路线图

基于 COCOMO II 模型的估算参考，系统开发划分为五个递进的阶段，总计预计 20–31 人月。第一阶段为 MVP（4–6 人月），交付基于 Python 的本地 WebSocket 中转服务和 EV3 基础电机控制 Scratch 扩展积木。第二阶段聚焦完整传感器支持（3–5 人月）。第三阶段建设数据采集系统（4–6 人月）。第四阶段集成 AI 能力与课程模块（5–8 人月）。第五阶段实现规模化部署支持（4–6 人月）。

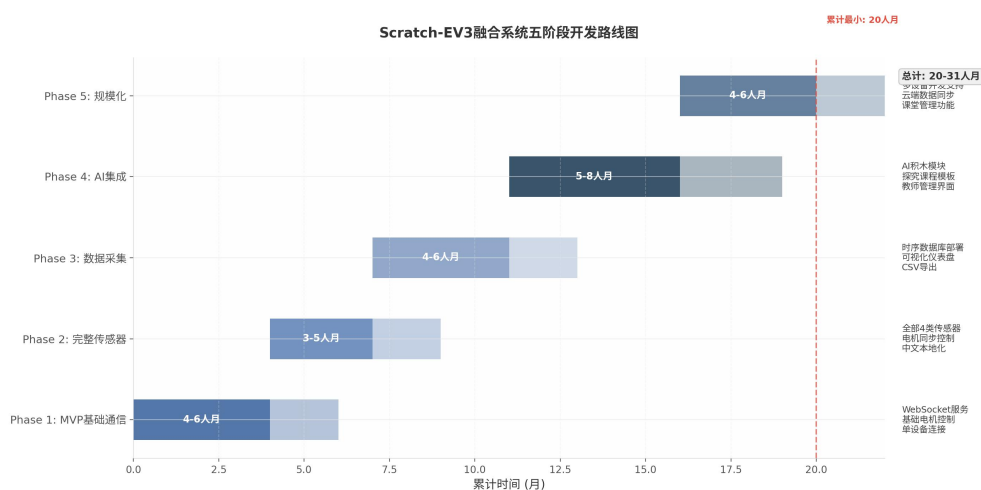


图 13: 五阶段开发路线图

8.3.2 教室级投入估算

以 16 名学生分为 8 组、每组 1 套 EV3 机器人为基准，系统评估了三档硬件采购方案。利用现有 EV3 方案仅需\$1,480 的增量投入（主要为 Raspberry Pi 终端和网络设备），适用于已拥有 EV3 设备的学校。二手采购方案总投入\$3,000。新购 SPIKE Prime 方案投入\$5,040。除一次性硬件投入外，项目还需考虑年度维护成本，开源软件的隐性成本年度总计约 0.5-1.0 个全职人力当量。

8.3.3 开源许可与知识产权

融合系统涉及的核心开源组件及其许可证兼容性分析如下。Scratch GUI/VM 采用 BSD-3-Clause 许可证，属于宽松型许可证。ev3dev 生态系统的许可证组合较为复杂：核心组件 brickd 采用 GPL-2.0，Python 绑定采用 MIT。Pybricks 采用 MIT 许可证。LEGO 商标的使用规范是知识产权合规的关键维度——LEGO Group 的 Fair Play 政策明确禁止第三方在域名、商业用途中使用 LEGO 商标。系统的开源许可策略建议为：以 BSD-3-Clause 发布 Scratch 扩展和前端代码，以 MIT 许可证发布 Python 中间件。

9. 结论与展望

本研究从系统架构、通信协议、传感器接口、数据流架构、融合设计、现有方案评估、教育应用可行性以及安全性与实施路径八个维度，对 Scratch 3.0 与 LEGO MINDSTORMS EV3 深度整合进行了系统性的可行性论证。研究结论表明，Scratch 与 EV3 的融合控制不仅在技术层面完全可行，而且在教育层面具有显著的价值创造潜力。

9.1 研究结论

9.1.1 技术可行性确认

从技术可行性角度，本研究通过逐层分析确认以下关键结论。第一，Scratch 3.0 的扩展架构（特别是 TurboWarp 的 Unsandboxed Extension 模式）能够支持完整的 EV3 硬件控制功能，消除官方扩展的沙箱延迟瓶颈。第二，基于

Python asyncio + WebSocket Server + JSON-RPC 2.0 的 Bridge Service 能够有效替代已弃用的 Scratch Link，并支持蓝牙与 WiFi 双模通信。第三，ev3dev sysfs 接口提供了对 50 余种传感器的标准化驱动支持，满足实时数据采集的硬件接口需求。第四，四层融合架构（前端交互层→虚拟机层→桥接服务层→硬件抽象层）设计合理，层间接口定义明确，能够支撑从单设备演示到教室级规模化部署的渐进式扩展。

9.1.2 教育价值论证

从教育价值角度，融合系统在以下三个层面展现出显著优势。在计算思维培养层面，元分析证据 ($d = 0.67$, CI: 0.40-0.95) 表明物理计算整合对学生 STEM 学业成绩具有中等偏大的正向效应，融合系统在“抽象”维度的培养覆盖度达到 9.5/10，显著高于纯 Scratch 编程 (7.0/10) 和纯 EV3 编程 (4.0/10)。在课程整合层面，融合系统的“智能实验平台”定位可直接对接中国 2022 信息科技课程标准、美国 NGSS 和 ISTE 学生标准三大权威框架。在教学实践层面，三阶段递进式课程模块（入门 6-8 课时、进阶 8-10 课时、高级 10-12 课时）和 4P 教学法确保了教学实施的可操作性。

9.1.3 关键创新发现

本研究进一步揭示三个具有战略意义的跨维度发现。第一个发现是“协议真空”创新窗口——Scratch Link 于 2024 年 5 月正式弃用与 EV3 于 2021 年 6 月停产的时间巧合，创造了一个无向后兼容约束的技术创新空间，使得全新协议设计成为可能。第二个发现是“教育物理数据”新品类——物理计算教育领域的数据标准空白为定义新的数据品类提供了学术发表机会。第三个发现是“架构即教学隐喻”——四层融合架构本身（前端可视化→虚拟机抽象→桥接服务→硬件物理）构成了计算思维层次化教学的天然隐喻。

9.2 实施建议

9.2.1 双轨推进策略

基于上述结论，本研究建议采用"开源核心+商业服务"的双轨推进策略。开源核心（包括 Scratch EV3Pro 扩展、Bridge Service 中间件和 EV3 后端服务）以 BSD-3-Clause/MIT 许可证发布，吸引社区贡献者参与开发和维护；商业服务（包括教师培训、课程资源包和技术支持）通过教育机构授权模式实现可持续发展。项目预计开发工作量 20-31 人月，单教室部署成本 1,480-5,040 美元，年度维护成本约 0.5-1.0 个全职人力当量。

9.2.2 阶段性里程碑

建议将项目实施划分为三个里程碑阶段。短期目标（6 个月内）完成 MVP 开发，交付基于 Python 的本地 WebSocket 中转服务和 EV3 基础电机控制 Scratch 扩展积木，在 1-2 所合作学校开展试点教学。中期目标（12 个月内）完成完整传感器支持和基础数据采集功能，形成可复用的课程模块库，覆盖 5-10 所学校。长期目标（24 个月内）集成 AI 能力和规模化部署支持，建立可持续的社区生态和商业运营模式，覆盖 50 所以上学校。

9.3 研究局限与未来方向

9.3.1 当前局限

本研究存在以下局限，需在后续工作中予以关注。第一，硬件性能约束——EV3 的 300MHz ARM9 处理器和 64MB RAM 对高频数据采集和多任务处理构成硬性上限，在 4 电机+4 传感器满配置下，传感器数据更新率约 10Hz。第二，蓝牙干扰风险——教室环境中 20-30 组设备同时运行时的 38%干扰概率仍需通过双模通信架构进行缓解。第三，实证验证缺失——本研究为架构层面的可行性分析，尚未通过大规模对照实验验证融合系统的实际教学效果。

9.3.2 未来研究方向

未来研究可从以下三个方向深入展开。第一，大规模教学实验——设计随机对照实验（RCT），比较融合系统与传统 Scratch/EV3 分离教学模式对学生计算思维发展的影响，目标样本量不少于 200 名学生。第二，多平台扩展——将四层融合架构迁移至 SPIKE Prime、Raspberry Pi 和 Arduino 等平台，验证架构的通用性和可移植性。第三，AI 深度整合——探索将边缘 AI 推理能力引入 EV3 后端（如通过 Coral USB Accelerator），实现传感器数据的本地智能处理，降低对云端资源的依赖。

9.3.3 结语

Scratch 与 EV3 的融合控制不仅是一个技术整合项目，更是面向 STEM 教育的一个新品类定义。当 EV3 从"机器人"变成"可编程传感器平台"，当 Scratch 从"游戏动画工具"变成"实验设计界面"，两者都发生了质变——而这种质变正是教育技术创新的本质：不是简单的工具叠加，而是通过深度融合创造全新的学习可能性。在技术可行性已经确认、教育价值已经论证的前提下，剩下的唯一问题是：我们何时开始行动？

感谢阅读

Scratch 与 EV3 融合控制系统可行性研究报告

基于开源架构的实时数据采集与教育应用分析 | 2025 年 12 月